

● 오라클 6일차

● ANSI - SQL join 문법

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e, hr.departments d
- WHERE e.department_id = d.department_id;
 - 기존 조인은 연결하는 컬럼을 찾아 연결
- NATURAL JOIN - 서로다른 테이블의 동일한 이름의 모든 기준의 연결고리를 (컬럼으로) 조인조건술어로 연결
 - - equi join의 형식으로 조인조건 술어를 자동으로 만들어줌
 - SELECT e.employee_id, e.salary, d.department_name
 - FROM hr.employees e NATURAL JOIN hr.departments d
 - 이로인해 틀린 컬럼도 연결 / 연결되는 컬럼의 데이터 타입이 동일해야 된다.
- -- NATURAL JOIN 또는 Join using 사용시 내부적으로 사용되는 조인조건술어에 해당하는 기준 컬럼 앞 테이블 이름 또는 별칭 사용 x
- SELECT d.department_name, d.department_id, d.manager_id, location_id, l.city
- FROM hr.departments d NATURAL JOIN hr.locations l;
- -- JOIN USING - equi join
- 내부적으로 조인조건을 수행해야할 기준컬럼지정
- 조인의 기준 컬럼이 되는 컬럼은 별칭을 사용해서는 안된다.
- SELECT e.employee_id, e.salary, d.department_name, l.city
- FROM hr.employees e join hr.departments d
- USING (department_id)
- JOIN hr.locations l
- USING (location_id)
- WHERE department_id IN (20,30,50);
- JOIN ON
- equi join, inner join
- ON절을 이용해서 조인조건 술어를 직접 만들어 사용
- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e, hr. department d
- WHERE e.department_id = d.department_id;
- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e JOIN hr.departments d
- ON e.department_id = d.department_id;
- SELECT e.employee_id, e.salary, d.department_name

- FROM hr.employees e INNER JOIN hr.departments d
- ON e.department_id = d.department_id;

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e JOIN hr.departments d
- ON e.department_id = d.department_id
- JOIN hr.locations l
- ON d.location_id = l.location_id;

- OUTER JOIN
- LEFT , RIGHT 를 이용해서 사용
- 양쪽의 경우 FULL을 사용하면 되지만 좋지 않음

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e , hr.departments d
- WHERE e.department_id = d.department_id(+);

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e LEFT OUTER JOIN hr.departments d
- ON e.department_id = d.department_id;

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e RIGHT OUTER JOIN hr.departments d
- ON e.department_id = d.department_id;

- cartesian product
- CROSS JOIN
- 두 테이블의 모든 행을 서로 조합하여 가능한 모든 경우의 수를 반환하는 SQL 조인 방식

- SELECT e.employee_id, e.salary, d.department_name
- FROM hr.employees e, hr.departments d;

- SELECT e.employee_id e.salary, d.department_name
- FROM hr.employees e CROSS JOIN hr.departments d

- /* 문제 36 2006년도에 입사한 직원들의 부서이름별 급여의 총액, 평균을 출력
- 1) 오라클 전용
- 2) ANSI 표준
-
- 문제 37 직원들의 last_name, salary, grade_level, department_name을 출력하는데
- last_name에 a문자가 2개 이상 포함되어 있는 직원들을 출력하세요.
- 1) 오라클 전용
- 2) ANSI 표준
-

- 문제 38 담당 관리자보다 먼저 입사한 사원의 이름과 입사일 및 해당 관리자의 이름과 입사일 출력하세요
- 1) 오라클 전용
- 2) ANSI 표준 */

```
SELECT d.department_name,
       SUM(e.salary) AS 급여총액,
       ROUND(AVG(e.salary),2) AS 급여평균
FROM hr.employees e, hr.departments d
WHERE e.department_id = d.department_id
AND TO_CHAR(e.hire_date, 'YYYY') = '2006'
GROUP BY d.department_name;
```

```
SELECT d.department_name,
       SUM(e.salary) AS 급여총액,
       ROUND(AVG(e.salary),2) AS 급여평균
FROM hr.employees e JOIN hr.departments d
ON e.department_id = d.department_id
AND TO_CHAR(e.hire_date, 'yyyy') = '2006'
GROUP BY d.department_name;
```

```
SELECT e.last_name, e.salary, j.grade_level, d.department_name
FROM hr.employees e, hr.departments d, hr.job_grades j
WHERE e.department_id = d.department_id
AND e.salary BETWEEN j.lowest_sal AND j.highest_sal
AND LOWER(e.last_name) LIKE '%a%a%';
```

```
SELECT e.last_name, e.salary, j.grade_level, d.department_name
FROM hr.employees e JOIN hr.departments d
ON e.department_id = d.department_id
JOIN hr.job_grades j
on e.salary BETWEEN j.lowest_sal AND j.highest_sal
AND LOWER(e.last_name) LIKE '%a%a%';
```

```
SELECT e.last_name AS 사원이름, e.hire_date AS 사원입사일, m.last_name
AS 관리자이름, m.hire_date AS 관리자입사일
FROM hr.employees e, hr.employees m
WHERE e.manager_id = m.employee_id
AND e.hire_date < m.hire_date;
```

```
SELECT e.last_name AS 사원이름, e.hire_date AS 사원입사일, m.last_name
AS 관리자이름, m.hire_date AS 관리자입사일
FROM hr.employees e JOIN hr.employees m
ON e.manager_id = m.employee_id
AND e.hire_date < m.hire_date;
```

- 서브쿼리
 - SQL문 안에 SELECT문을 서브쿼리라고 한다.

- SELECT문 안에서 서브쿼리는 괄호()로 묶어야 된다.
- -- 110번 사원의 급여보다 더 많은 급여를 받는 사원?
- SELECT *
- FROM hr.employees e
- WHERE salary > (SELECT salary FROM hr.employees WHERE employee_id = 110);
- 중첩서브쿼리
 - main(outer) query, sub(inner) query
 - 서브쿼리 먼저 수행 / 서브쿼리에서 수행한 값을 가지고 메인 쿼리 수행
 - 서브쿼리에서 리턴되는 값이 단일 값이 아닐 경우에는 에러발생
- 단일행 서브쿼리
 - 서브쿼리의 결과가 단일값이 나오는 서브쿼리
 - 단일행 비교 연산자 (= , > , >= , < , <= , <> , != , ^=)
 - /* 문제 39 110번 사원의 job_id와 동일한 사원들 중에 110번 사원의 급여보다 더 많이 받는 사원들의 정보를 추출하세요.
 - 문제 40 최고 급여를 받는 사원들의 정보를 출력해주세요. */
 - SELECT *
 - FROM hr.employees
 - WHERE job_id = (SELECT job_id FROM hr.employees WHERE employee_id = 110)
 - AND salary > (SELECT salary FROM hr.employees WHERE employee_id = 110);
 - SELECT *
 - FROM hr.employees
 - WHERE salary = (SELECT MAX(salary) FROM hr.employees);
 - SELECT MAX(sum(salary))
 - FROM hr.employees
 - GROUP BY department_id;
 - SELECT MAX(sum(salary))
 - FROM hr.employees
 - GROUP BY department_id;
 - SELECT department_id, sum(salary)
 - FROM hr.employees
 - GROUP BY department_id
 - HAVING sum(salary) = (SELECT MAX(sum(salary)) FROM hr.employees GROUP BY department_id);
 - group by (그룹함수)에서는 두번 중첩할 경우 에러가 발생하기 때문에 개별 컬럼을 사용할 수 없음 > 해결 방법으로 서브쿼리를 이용

- SELECT *
- FROM hr.employees
- WHERE salary = (SELECT salary FROM hr.employees WHERE job_id = 'IT_PROG');
- 서브쿼리에서 추출되는 값이 여러 개로 오류 발생
- 여러 행 연산자
 - 서브쿼리의 결과가 여러개의 값이 나오는 서브쿼리를 여러행 서브쿼리라고 한다.
 - 여러행 비교연산자 사용 (IN, ANY, ALL)
 - SELECT *
 - FROM hr.employees
 - WHERE salary IN (SELECT salary
 - FROM hr.employees
 - WHERE job_id = 'IT_PROG');
 - ANY = OR - IN
 - ALL = AND라는 의미 - MIN, MAX로 변경 가능
- OR 진리표
 - TRUE OR TRUE = True
 - TRUE OR FALSE = True
 - TRUE OR NULL = True
 - FALSE OR NULL = Null
- AND 진리표
 - TRUE AND TRUE = True
 - TRUE AND FALSE = False
 - TRUE AND NULL = Null
 - FALSE AND NULL = False
 - AND 진리표로 인해 NOT IN 연산자의 값이 조회가 안됨
 - 해결방법으로는 IS NOT NULL로 NULL를 제외
- Correlate subquery (상호관련 서브쿼리)
 - main(outer) query 먼저 수행
 - 첫 번째 행을 후보행으로 잡고 후보행값을 서브쿼리에 전달
 - 후보행값을 사용해서 서브쿼리 수행
 - 서브쿼리 결과값을 사용 후 후보행과 비교 참이면 후보행을 결과집합 저장 거짓이면 후보행을 버림
 - 두 번째 행을 후보행으로 잡고 2, 3, 4번을 반복 수행 메인 쿼리에 행의 수만큼 반복
- InLine View
 - 가상테이블 / From절 괄호 안에 Select 문을 (서브쿼리) InLine View
 - /* 자신의 부서 평균 급여보다 많이 받는 사원은 ?*/
 - SELECT *
 - FROM hr.employees e
 - WHERE salary > (SELECT AVG(salary)
 - FROM hr.employees
 - WHERE department_id = e.department_id);

- SELECT e2.*
- FROM(
 - SELECT department_id, ROUND(avg(salary),2) avg_sal
 - FROM hr.employees
 - GROUP BY department_id) e1, hr.employees e2
 - WHERE e1.department_id = e2.department_id
 - AND e2.salary > e1.avg_sal;